

# Exploring the Design Space for a Pipeline Operator

Document #: P2672R0  
Date: 2022-10-13  
Project: Programming Language C++  
Audience: EWG  
Reply-to: Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>

## Contents

---

- [1 Various Designs for Pipelines](#)
  - [1.1 Left-Threading](#)
  - [1.2 Inverted Invocation](#)
  - [1.3 Placeholder](#)
  - [1.4 Language Bind](#)
- [2 Comparing the Designs](#)
  - [2.1 Rejecting the Inverted Application Model](#)
  - [2.2 To Placeholder or Not To Placeholder](#)
  - [2.3 Placeholder or Language Bind](#)
- [3 Design Details](#)
  - [3.1 Right-hand side restrictions](#)
    - [3.1.1 Unevaluated Contexts](#)
    - [3.1.2 Conditionally-Evaluated Contexts](#)
    - [3.1.3 Even-more-conditionally-evaluated Contexts](#)
    - [3.1.4 Nested Pipeline Expressions](#)
    - [3.1.5 Everything else is okay](#)
  - [3.2 Operator Precedence](#)
  - [3.3 Multiple Placeholders](#)
- [4 Placeholder Lambdas](#)
  - [4.1 Partial Application](#)
- [5 Choice of Placeholder](#)
- [6 Disposition](#)
  - [6.1 Is This Worth It?](#)
- [7 References](#)

## § 1 Various Designs for Pipelines

---

[[P2011R0](#)] proposed a new, non-overloadable binary operator `|>` such that the expression

```
x |> f(y)
```

was defined to be evaluated as:

```
f(x, y)
```

without any intermediate `f(y)` expression. The rules of this operator were fairly simple: the right-hand side had to be a call expression, and the left-hand expression was inserted as the first argument into the right-hand call.

But there are other potential designs for such an operator. The goal of this paper is to go over all of the possibilities and attempt to weigh their pros and cons. It would also be useful background to read the JavaScript proposal for this operator [[javascript.pipeline](#)], which both provides a lot of rationale for the feature and goes through a similar exercise.

In short, there are four potential designs. I'll first present all four and then discuss the pros and cons of each: [Left-Threading](#), [Inverted Invocation](#), [Placeholder](#), and [Language Bind](#).

## § 1.1 Left-Threading

The left-threading model was what was proposed in [[P2011R0](#)] (and is used by Elixir). In `x |> e`, `e` has to be a call expression of the form `f(args...)` and the evaluation of this operator is a rewrite which places the operand on the left-hand of the pipeline as the first call argument on the right-hand side. This is similar to how member function invocation works (especially in a deducing `this` world).

For example:

| Code                       | Evaluation                                                      |
|----------------------------|-----------------------------------------------------------------|
| <code>x  &gt; f(y)</code>  | <code>f(x, y)</code>                                            |
| <code>x  &gt; f()</code>   | <code>f(x)</code>                                               |
| <code>x  &gt; f</code>     | ill-formed, because <code>f</code> is not a call expression     |
| <code>x  &gt; f + g</code> | ill-formed, because <code>f + g</code> is not a call expression |

## § 1.2 Inverted Invocation

In the inverted invocation model (which is used by F#, Julia, Elm, and OCaml), `|>` is an inverted call invocation. `x |> f` is defined as `f(x)`. The right-hand side can be arbitrarily involved, and `|>` would be sufficiently low precedence to allow this flexibility:

| Code                       | Evaluation              |
|----------------------------|-------------------------|
| <code>x  &gt; f(y)</code>  | <code>f(y)(x)</code>    |
| <code>x  &gt; f()</code>   | <code>f()(x)</code>     |
| <code>x  &gt; f</code>     | <code>f(x)</code>       |
| <code>x  &gt; f + g</code> | <code>(f + g)(x)</code> |

## § 1.3 Placeholder

In the placeholder model (which is used by Hack), the right-hand side of `|>` is an arbitrary expression that must contain at least one placeholder. In Hack, that placeholder is `$$`. But for the purposes of this paper, I'm going to use `%` (see [here](#) for discussion on placeholder choice). The pipeline operator evaluates as if replacing all instances of the placeholder with the left-hand argument:

| Code                             | Evaluation                 |
|----------------------------------|----------------------------|
| <code>x  &gt; f</code>           | ill-formed, no placeholder |
| <code>x  &gt; f(y)</code>        | ill-formed, no placeholder |
| <code>x  &gt; f(%, y)</code>     | <code>f(x, y)</code>       |
| <code>x  &gt; f(y, %)</code>     | <code>f(y, x)</code>       |
| <code>x  &gt; y + %</code>       | <code>y + x</code>         |
| <code>x  &gt; f(y) + g(%)</code> | <code>f(y) + g(x)</code>   |
| <code>x  &gt; co_await %</code>  | <code>co_await x</code>    |
| <code>x  &gt; f(1, %, %)</code>  | <code>f(1, x, x)</code>    |

## § 1.4 Language Bind

Consider again the expression `x |> f(y, %)` above. What role does the `f(y, %)` part serve? I don't want to call it a sub-expression since it's not technically a distinct operand here. But conceptually, it at least kind of is. And as such the role that it serves here is quite similar to:

```
std::bind(f, y, _1)(x)
```

With two very notable differences:

- The `bind` expression is limited to solely functions, which the placeholder pipeline is not (as illustrated earlier). But more than that, the `bind` expression is limited to the kinds of functions we can pass as parameters, which `f` need not be (e.g. `std::apply` or `std::visit`, or any other function template)
- The `bind` expression has to capture any additional arguments (the bound arguments) because, as a library facility, it is not knowable when those arguments will actually be used. How expensive is capturing `f` and `y`? But with the placeholder expression, we don't need to capture anything, since we know we're immediately evaluating the expression.

That said, the not-quite-expression `f(y, %)` is conceptually a lot like a unary function.

With that in mind, the language bind model is the inverted invocation model except also introducing the ability to use placeholders to introduce a language bind (sort of like partial application). That is:

| Code                         | Evaluation           |
|------------------------------|----------------------|
| <code>x  &gt; f</code>       | <code>f(x)</code>    |
| <code>x  &gt; f(y)</code>    | <code>f(y)(x)</code> |
| <code>x  &gt; f(%, y)</code> | <code>f(x, y)</code> |
| <code>x  &gt; f(y, %)</code> | <code>f(y, x)</code> |
| <code>x  &gt; y + %</code>   | <code>y + x</code>   |

| Code                             | Evaluation               |
|----------------------------------|--------------------------|
| <code>x  &gt; f(y) + g(%)</code> | <code>f(y) + g(x)</code> |
| <code>x  &gt; f + g</code>       | <code>(f + g)(x)</code>  |
| <code>x  &gt; co_await %</code>  | <code>co_await x</code>  |
| <code>x  &gt; f(1, %, %)</code>  | <code>f(1, x, x)</code>  |

## § 2 Comparing the Designs

Now let's try to determine which of these we should pursue.

### § 2.1 Rejecting the Inverted Application Model

Of these, the inverted invocation model (or the F# model) the simplest to understand, specify, and implement. However, for C++ in particular, it is also the least useful.

It actually does happen to still work for Ranges:

```
// this expression
r |> views::transform(f) |> views::filter(g)

// evaluates as
views::filter(g)(views::transform(f)(r))
```

Nobody would write the latter code today (probably), but it is valid - because `views::transform(f)` and `views::filter(g)` do give you unary function objects. But in order for that to be the case, additional library work has to be done - and it's precisely that library work that we wrote [\[P2011R0\]](#) to avoid.

Without ready-made unary functions, we'd have to write lambdas, and our lambdas are not exactly terse:

```
r |> [=](auto&& r){ return views::transform(FWD(r); f); }
  |> [=](auto&& r){ return views::filter(FWD(r); g); }
```

Nor are our binders which additionally work on only a restricted set of potential callables:

```
r |> std::bind_back(views::transform, f)
  |> std::bind_back(views::filter, g)
```

And none of this avoids the unnecessary capture that this model would require. As such, it's the easiest to reject.

### § 2.2 To Placeholder or Not To Placeholder

With placeholders, we get significantly more flexibility. There are situations where the the parameter we want to pipe into isn't the first parameter of the function. There are multiple such examples in the standard library:

- In Ranges, `r1 |> zip_transform(f, %, r2)`. The function is the first parameter, but range pipelines are going to be built up of ranges.

- Similarly, `some_tuple |> apply(f, %)` and `some_variant |> visit(f, %)` fall into the same boat: the function is the first parameter, but the “subject” of the operation is what you want to put on the left.

Additionally there are going to be cases where the expression we want to pipe the left-hand argument into isn’t a function call, the fact that it could be anything allows for a wide variety of possibilities. Or that you could pipe into the expression multiple times.

Those are large plusses.

On the flip side, using a placeholder is necessarily more verbose than the left-threading model, by just a single character for unary adaptors (`|> f()` vs `|> f(%)`, the parentheses would be required in the left-threading model) and by three characters per invocation (`%`, assuming you put a space between function arguments) for multiple arguments:

```
// left-threading
r |> views::transform(f)
  |> views::filter(g)
  |> views::reverse();

// placeholder
r |> views::transform(%, f)
  |> views::filter(%, g)
  |> views::reverse(%)
```

Now, while the placeholder allows more flexibility in expressions, for certain kinds of libraries (like Ranges and the new Sender/Receiver model), the common case (indeed, the overwhelmingly common case) is to pipe into the first argument. With the left-threading model, we have no syntactic overhead as a result and don’t lose out on much. With the placeholder model, we’d end up with a veritable sea of `meow(%)` and `meow(%, arg)s`.

Since left-threading is the one thing we can do today (by way of `|` and library machinery), it would arguably be more accurate to say that the placeholder model is four characters per call longer than the status quo:

```
// status quo
| views::transform(f)

// left-threading
|> views::transform(f)

// placeholder
|> views::transform(%, f)
```

Flexibility vs three extra characters may seem like a silly comparison, but ergonomics matters, and we do have libraries that are specifically designed around first-parameter-passing. It would be nice to not have to pay more syntax when we don’t need to.

On the whole though the argument seems to strongly favor placeholders, and if anything exploring a special case of the pipeline operator that does left-threading only and has a more restrictive right-hand side to avoid potential bugs. That might still allow the best of both worlds.

A recent [Conor Hoekstra talk](#) has a nice example that I'll present multiple different ways (in all cases, I will not use the `|` from Ranges).

With left-threading, the example looks like:

```
auto filter_out_html_tags(std::string_view sv) -> std::string {
    auto angle_bracket_mask =
        sv |> std::views::transform([](char c){ return c == '<' or c == '>'; });

    return std::views::zip_transform(
        std::logical_or(),
        angle_bracket_mask,
        angle_bracket_mask |> rv::scan_left(std::not_equal_to{}))
        |> std::views::zip(sv)
        |> std::views::filter([](auto t){ return not std::get<0>(t); })
        |> std::views::values()
        |> std::ranges::to<std::string>();
}
```

Notably here we run into both of the limitations of left-threading: we need to pipe into a parameter other than the first and we need to pipe more than once. That requires introducing a new named variable, which is part of what this facility is trying to avoid the need for. This is not a problem for either of the placeholder-using models, as we'll see shortly.

With the placeholder-mandatory model, we don't need that temporary, since we can select which parameters of `zip_transform` to pipe into, and indeed we can pipe twice (I'll have more to say about nested placeholders later):

```
auto filter_out_html_tags(std::string_view sv) -> std::string {
    return sv
        |> std::views::transform(%, [](char c){ return c == '<' or c == '>'; })
        |> std::views::zip_transform(std::logical_or{}, %, % |> rv::scan_left(%,
            std::not_equal_to{}))
        |> std::views::zip(%, sv)
        |> std::views::filter(%, [](auto t){ return not std::get<0>(t); })
        |> std::views::values(%)
        |> std::ranges::to<std::string>(%)
}
```

With the language bind model, we can omit the two uses of `(%)` for the two unary range adaptors:

```
auto filter_out_html_tags(std::string_view sv) -> std::string {
    return sv
        |> std::views::transform(%, [](char c){ return c == '<' or c == '>'; })
        |> std::views::zip_transform(std::logical_or{}, %, % |> rv::scan_left(%,
            std::not_equal_to{}))
        |> std::views::zip(%, sv)
        |> std::views::filter(%, [](auto t){ return not std::get<0>(t); })
        |> std::views::values
        |> std::ranges::to<std::string>;
}
```

And with the introduction of a dedicated operator for left-threading, say `\>`, we can omit four more instances of placeholder:

```
auto filter_out_html_tags(std::string_view sv) -> std::string {
    return sv
        \> std::views::transform([](char c){ return c == '<' or c == '>'; })
        |> std::views::zip_with(std::logical_or{}, %, % \>
            rv::scan_left(std::not_equal_to{}))
        \> std::views::zip(sv)
        \> std::views::filter([](auto t){ return not std::get<0>(t); })
        |> std::views::values
        |> std::ranges::to<std::string>;
}
```

The difference the various placeholder models is about counting characters. For unary functions, can we write `x |> f` or do we have to write `x |> f(%)`? And then for left-threading, do we have to write `x |> f(%, y)` or can we avoid the placeholder with a dedicated `x \> f(y)`? Overall, the last solution (language bind with `\>`) is 18 characters shorter than the placeholder solution, simply by removing what is arguably syntactic noise.

To be honest though, regardless of those 18 characters, the thing that annoys me the most in this example is the lambda. More on that later.

## § 2.3 Placeholder or Language Bind

These are the two most similar models, so let's just compare them against each other using a representative example:

| Placeholder                                                                                                                                                 | Language Bind                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>auto v =     r  &gt; views::transform(%, f)      &gt; views::filter(%, g)      &gt; views::reverse(%)      &gt; ranges::to&lt;std::vector&gt;(%)</pre> | <pre>auto v =     r  &gt; views::transform(%, f)      &gt; views::filter(%, g)      &gt; views::reverse      &gt; ranges::to&lt;std::vector&gt;</pre> |

When the range adaptor is binary (as in `transform` or `filter` or many others), the two are equivalent. We use a placeholder (`%` in this case) for the left-hand side and then provide the other argument manually. No additional binding occurs.

But when the range adaptor is unary, in the placeholder (Hack) model, we still have to use a placeholder. Because we always have to use a placeholder, that's the model. But in the language bind model, a unary adaptor is already a unary function, so there's no need to use language bind to produce one. It just works. In the case where we already have a unary function, the language bind model is three characters shorter - no need to write the `(%)`.

Consider this alternative example, which would be the same syntax in both models:

```

auto squares() -> std::generator<int> {
    std::views::iota(0)
    |> std::views::transform(%, [](int i){ return i * i; })
    |> co_yield std::ranges::elements_of(%);
}

```

Admittedly not the most straightforward way to write this function, but it works as an example and helps demonstrate the utility and flexibility of placeholders. Now, let's talk about what this example means in the respective models.

In the placeholder model, this is a straightforward rewrite into a different expression - because the placeholder model is always a rewrite into a different expression. Even if that expression is a `co_yield`.

But in the language bind model, this becomes a little fuzzier. If we say that `co_yield std::ranges::elements_of(%)` is effectively a language bind (even if we side-step the question of captures since we know we're immediately evaluating), that sort of has to imply that the `co_yield` happens in the body of some new function right? But `co_yield` can't work like that, it has to be invoked from `squares` and not from any other internal function. It's not like we actually need to construct a lambda to evaluate this expression, but it does break the cleanliness of saying that this is just inverted function invocation.

Language bind is a more complex model than placeholders and requires a little hand-waving around what exactly we're doing, for a benefit of three characters per unary function call. Is it worth it?

## § 3 Design Details

At this point, I think it's clear that placeholders are superior to left-threading. Just significantly more flexibility. The questions are really what we want to do about no placeholders (should `x |> f` be valid, as per the language bind model, or invalid, as per the regular placeholder model) and whether we should do something the gain back the smaller syntax of the left-threading model (should `x \> f()` or something to that effect be valid and mean `f(x)`). Let's hold off on those for now and deal with a few other issues about placeholders that definitely need to be addressed.

### § 3.1 Right-hand side restrictions

Generally, the model for `A |> E` is that `E` is evaluated with `A` substituted into the `%` except that `A` is evaluated first. `A` is evaluated first largely because it clearly looks like it is - it's written first. This is similar to the argument made in [\[P0145R3\]](#) for changing the evaluation order of expressions for C++17.

There are a wide variety of expressions that we can use for `E` where this ends up not being a problem - either because the current evaluation order is unspecified or changing it doesn't cause problems. For instance:

| Expression                 | Evaluated as       | Notes                                                                                    |
|----------------------------|--------------------|------------------------------------------------------------------------------------------|
| <code>A  &gt; B + %</code> | <code>B + A</code> | Currently unspecified, okay to just evaluate <code>A</code> first                        |
| <code>A  &gt; B(%)</code>  | <code>B(A)</code>  | Currently <code>B</code> evaluated first, but it's okay to evaluate <code>A</code> first |



| Expression        | Evaluated as | Notes                                                          |
|-------------------|--------------|----------------------------------------------------------------|
| A  > B[%]         | B[A]         | Currently B evaluated first, but it's okay to evaluate A first |
| A  > B(%)  > C(%) | C(B(A))      | Currently, C then B then A, but here would be the reverse      |

But, there are several situations where changing the evaluation order like this would be, at best, very surprising. It's worth going over such cases.

### § 3.1.1 Unevaluated Contexts

If substituting into the placeholder gives you an unevaluated expression (exclusively unevaluated expressions, since there can be [multiple placeholders](#)), the expression is ill-formed. For instance:

```
int f();

auto v = f() |> std::vector<decltype(%)>{};
```

If we substitute into the placeholder, we get `std::vector<decltype(f())>{}`, which doesn't actually evaluate `f()`. But unlike this direct rewrite, where `f()` is clearly unevaluated by nature of lexically appearing inside of `decltype`, in the pipelined expression it sure looks like `f()` is actually evaluated.

However, there is an interesting use-case for wanting to allow unevaluated contexts. Consider [Boost.Mp11](#). It's a very useful metaprogramming library, that suffers only from syntax limitations. For instance, the reference has an example implementation of `tuple_cat` that does:

```
// inner
using list1 = mp_list<
    mp_rename<typename std::remove_reference<Tp>::type, mp_list>...>;
using list2 = mp_iota_c<N>;
using list3 = mp_transform<mp_fill, list1, list2>;
using inner = mp_apply<mp_append, list3>;

// outer
using list4 = mp_transform<F, list1>;
using outer = mp_apply<mp_append, list4>;
```

But if we could use `|>` with a placeholder, even if these things aren't... entirely expressions, this could be:

```
// inner
using list1 = mp_list<
    mp_rename<typename std::remove_reference<Tp>::type, mp_list>...>;
using inner = mp_transform<mp_fill, list1, mp_iota_c<N>> |>
    mp_apply<mp_append, %>;

// outer
using outer = mp_transform<F, list1> |> mp_apply<mp_append, %>;
```

The same argument for the advantages of linearity for normal expressions apply just as well here. But this might be a bit too far, and would require rethinking probably too much of the grammar.

I think the right rule here is: if all the placeholders are only used in unevaluated contexts, the pipeline expression is ill-formed. No `decltype`, `requires` (note that using a pipeline expression as part of the *expression* of a *simple-requirement* or *compound-requirement* is fine - it's just that piping *into* a requirement is not), `sizeof`, etc.

### § 3.1.2 Conditionally-Evaluated Contexts

Following the same theme, consider the following expressions:

| Expression                           |
|--------------------------------------|
| <code>f()  &gt; g() or %</code>      |
| <code>f()  &gt; g() and %</code>     |
| <code>f()  &gt; g() ? % : h()</code> |

In these right-hand expressions, the placeholder is only sometimes evaluated. With `g() or f()`, we typically only evaluate `f()` if `g()` is `false`. Here, `f() |> g() or %` would have to unconditionally evaluate `f()` and then also unconditionally evaluate `g()`.

Is that actually what the user wanted, or did they expect the typical short-circuiting behavior more typically associated with this operators?

To avoid confusion, placeholders shall not appear exclusively as the second operand of logical or/and or as the second or third operand of the conditional operator. Of course, we need to handle arbitrary expressions there - `f() |> g() or (% + 1)` too, and so forth. This isn't really a huge loss of functionality, but it does seem like a big alleviation in potential confusion.

### § 3.1.3 Even-more-conditionally-evaluated Contexts

There are a few other expressions which are even more problematic than the above. Consider:

```
auto g = f() |> [=]{ return %; };
```

What does this mean? Does it evaluate `f()` or not? If so, would invoking `g` call `f` again or is the result cached and invoking `g` just directly returns it?

Arguably, this is simply meaningless. This isn't what the pipeline operator is for. If you want the invoke-and-cache functionality, that is directly expressible without any problem:

```
auto g = f() |> [r=%]{ return r; };
```

Using placeholders in the body of a lambda is disallowed, but as part of the initializer in an init-capture is fine.

The same principle will hold for pattern-matching when it's adopted [P1371R3]. Piping into the *expression* that `inspect` is inspecting is fine - any other part of an `inspect`-expression is off limits.

### § 3.1.4 Nested Pipeline Expressions

I expect the typical use of pipeline to be just chaining a bunch of operations together, linearly:

```
A |> B(%) |> C(%, D) |> co_await %;
```

Which evaluates as follows (except that the evaluation order is definitely A then B then C then D):

```
co_await C(B(A), D);
```

Now, what happens if we instead parenthesize part of the right-hand side:

```
A |> (B(%) |> C(%, D));
```

Here, the right-hand side of the first pipeline operator is all of `B(%) |> C(%, D)`, which contains two placeholders. The question is: how do we substitute into this?

Importantly, we *only* substitute into the left-hand operand (and thus only the left-hand operand participates in the rule for a placeholder being required). An example to help motivate this decision can be seen in Conor's example earlier:

```
sv
|> views::transform(%, [](char c){ return c == '<' or c == '>'; })
|> views::zip_transform(std::logical_or{}, %, % |> views::scan_left(%,
    std::not_equal_to{}))
```

The intent here is to zip two versions of the transformed sv: by itself, and then a `scan_left` version of it. This is a pretty reasonable thing to want to do, and also seems like a reasonable interpretation of what this means syntactically.

If I take a simpler version of the above:

```
A |> B(%) |> C(%, % |> D(%, E))

// ... evaluates as
B(A) |> C(%, % |> D(%, E))

// ... evaluates as (sort of, see discussion on multiple placeholders)
// the substitution of B(A) *only* goes into the left-hand operand of
// the other |>
C(B(A), B(A) |> D(%, E))

// ... evaluates as
C(B(A), D(B(A), E))
```

For completeness, there are two other possible interpretations.

Having a right-hand side use of `%` could be ill-formed, but this seems overly restrictive.

Or we could substitute into *both* sides. What would that look like? Consider the expression:

```
x |> f(%, y |> g(%, z))
```

If we substitute just into the left-hand side, we get what is almost certainly the intended meaning of the above:

```
f(x, g(y, z))
```

If we substitute into *both* sides, the initial substitution becomes:

```
f(x, y |> g(x, z))
```

In the placeholder model, this is ill-formed, since the remaining `|>` expression has no placeholder on the right-hand side. But in the language bind model, this would be valid and evaluate as:

```
f(x, g(x, z)(y))
```

This seems... very unlikely to be the desired meaning. And if it were, it could be easily expressed in a less cryptic way:

```
x |> f(%, g(%, z)(y))
```

In short, if the right-hand operand of a `|>` expression itself contains a placeholder, it is not substituted into. Effectively, it's a firewall. Only the left-hand operand is substituted into (if the left-hand operand has a placeholder, that is).

### § 3.1.5 Everything else is okay

Every other expression is fair game. All the unary and binary operators, the postfix expressions (including the named casts), `co_await`, `co_yield`, and `throw`.

There's a question as to whether it's worth supporting this:

```
int f() {  
    42 |> return %;  
}
```

It's easy enough to specify, but I doubt it's actually worthwhile to do, and having to write `return` (or `co_return`) first doesn't actually seem like that big a burden if the pipeline operator lets you go ahead and chain the entirety of the rest of the expression. Might even be good to explicitly *not* support piping into `return`. In any case, this paper only deals with expressions. So the above is ill-formed.

## § 3.2 Operator Precedence

One of the interesting things about this feature is how different the question of precedence is based on the choice of model.

For the [left-threading](#) and [inverted invocation](#) models, since `|>` functions as just another function call, it makes a lot of sense to treat it as a *postfix-expression*. That is how [\[P2011R0\]](#) [defined it](#), although there end up being potentially confusing interactions with unary operators that led to [\[P2011R1\]](#) moving it down a bit (see the [operator precedence section](#) there).

Either way, `|>` in these models has very high precedence (either equal to function call, or just below the unary operators).

But for the [placeholder](#) or [language bind](#) models, since `|>` isn't just a function call but rather a full expression rewrite, it makes sense to allow the right-hand “operand” to be any expression at all. That is, the precedence should be *extremely* low. Reprinting the precedence chart from [\[P2011R1\]](#):

| Precedence | Operator                                                                                                                                                                                                               |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1          | <code>::</code>                                                                                                                                                                                                        |
| 2          | <code>a++ a--</code><br><code>τ() τ{}</code><br><code>a() a[]</code><br><code>. -&gt;</code><br><i>– P2011R0 –</i>                                                                                                     |
| 3          | <code>++a --a</code><br><code>+a -a</code><br><code>! ~</code><br><code>(τ)</code><br><code>*a &amp;a</code><br><code>sizeof</code><br><code>co_await</code><br><code>new new[]</code><br><code>delete delete[]</code> |
| 4          | <code>. * -&gt; *</code>                                                                                                                                                                                               |
| 4.5        | <i>– P2011R1 –</i>                                                                                                                                                                                                     |
| 5          | <code>a*b a/b a%b</code>                                                                                                                                                                                               |
| 6          | <code>a+b a-b</code>                                                                                                                                                                                                   |
| 7          | <code>&lt;&lt; &gt;&gt;</code><br><i>– Elixir, F#, OCaml –</i>                                                                                                                                                         |
| 8          | <code>&lt;=&gt;</code>                                                                                                                                                                                                 |
| 9          | <code>&lt; &lt;=</code><br><code>&gt; &gt;=</code>                                                                                                                                                                     |
| 10         | <code>== !=</code>                                                                                                                                                                                                     |
| 11         | <code>&amp;</code>                                                                                                                                                                                                     |
| 12         | <code>^</code>                                                                                                                                                                                                         |
| 13         | <code> </code>                                                                                                                                                                                                         |
| 14         | <code>&amp;&amp;</code>                                                                                                                                                                                                |
| 15         | <code>  </code>                                                                                                                                                                                                        |
| 15.5       | <i>– JavaScript, Hack, Elm –</i>                                                                                                                                                                                       |
| 16         | <code>a?b:c</code><br><code>throw</code><br><code>co_yield</code><br><code>=</code><br><code>op=</code>                                                                                                                |
| 17         | <code>,</code>                                                                                                                                                                                                         |

That a placeholder `|>` should go, at least, below the logical operators goes without saying. The only question to my mind is whether it should be above or below assignment. And here, I think other languages make the correct choice. If we write something like:

```
v = x |> f(%) |> g(%) ;
```

I think there is a strong expectation that assignment has very low precedence and that the above evaluates as:

```
v = g(f(x)) ;
```

Rather than (if `|>` was even lower than assignment) as:

```
g(f(v = x)) ;
```

If the latter meaning was really intended, users can parenthesize `(v = x)` as usual.

## § 3.3 Multiple Placeholders

It's fairly clear what to do in the case of a single use of placeholder on the right hand side of `|>`: substitute. Doesn't matter whether the left-hand operand is an lvalue or rvalue, regardless of how complex the expression is. `x + y |> f(%)` is just `f(x + y)` (except that `x + y` is definitely evaluated before `f`)

But with multiple placeholders, it's not that simple. What should this mean:

```
auto fork = f() |> g(%, %);
```

Having it evaluate as `auto fork = g(f(), f());` would be pretty bad. At best, it's very surprising to have `f()` evaluate twice in code that only wrote it one time. At worst, this is just wrong. If the intent is to call `f()` twice, that should probably be more explicit in the code (even if sometimes it would be okay to do so).

That leaves evaluating `f()` one time and using the result multiple times. If `f()` is an lvalue, this isn't a problem. Passing the same lvalue multiple times is fine

But if `f()` is an rvalue, we have to make a choice for what to do. Let's call the result `r`. We could:

1. Use `r` as an lvalue every time: `g(r, r)`
2. Use `r` as an rvalue every time: `g(move(r), move(r))`
3. Use `r` as an lvalue every time but one and as an rvalue for the last one: `g(r, move(r))` (if the implementation evaluates function arguments from left-to-right)
4. Ill-formed if trying to substitute an rvalue multiple times.

Moving twice is problematic if `g` takes both parameters by value - one of them will be moved-from. It's not even great if `g` takes both parameters by rvalue reference. Even if that's technically what the user wrote (`f()` was an rvalue and they *did* want to pipe it twice), and even if that's the behavior that

`std::bind` has (`std::bind(f, _1, _1)(rv)` will actually move from the rvalue twice), I don't think it's what we should actually do.

Moving just the last is more efficient and actually safe, but now you have no idea if you can call `g(T&, T&&)` since the viability of this expression depends on implementation-defined evaluation order of function arguments. That just seems inherently not great.

Note that because the right-hand side need not be a call expression, the order of evaluation could well be defined. For instance, in `f() |> %(%)`, this would have to evaluate as `r(move(r))`. But in `f() |> % + %`, we have the same issue again.

I think that basically leaves either passing the evaluated left-hand argument as an lvalue multiple times or ill-formed. It's certainly potentially surprising, as `f() |> g(%)` and `f() |> g(%, %)` end up being quite different, but I think it's a defensible and practical choice. Alternatively, if we say it's ill-formed to pipe an rvalue multiple times, the user could always themselves implement multiple lvalue passing:

```
template <class T> auto as_lvalue(T&& t) -> T& { return (T&)t; }  
  
f() |> as_lvalue(%) |> g(%, %)
```

If `f()` were an lvalue, the `as_lvalue` “cast” is pointless, and we fork the result (as a single evaluation) to `g`. If `f()` were an rvalue, then it is lifted to an lvalue, and now the language rule would allow it to be forked (but now it's explicit in code, rather than implicit in the language).

I'm not sure this explicitness is worthwhile. Such use won't necessarily be common, but I don't think it will end up being exceedingly rare either.

In short, if there are multiple placeholders in the right-hand expression, then the left-hand expression is evaluated one time and substituted, as an lvalue, into each placeholder.

## § 4 Placeholder Lambdas

One reason I consider the language `bind` model attractive, despite the added complexity (both in having to handle `x |> f` in addition to `x |> f(%)` and also having to hand-wave around what `x |> co_yield %` means) is that it also offers a path towards placeholder lambdas. Allow me an aside.

There are basically three approaches that languages take to lambdas (some languages do more than one of these).

1. Direct language support for partial application
2. Placeholder expressions
3. Abbreviated function declarations

I'll illustrate what I mean here by using a simple example: how do you write a lambda that is a unary predicate which checks if its parameter is a negative integer?

For the languages that support partial application, that looks like:

| Expression            | Language |
|-----------------------|----------|
| <code>(&lt;0)</code>  | Haskell  |
| <code>(&lt;) 0</code> | F#       |

For the languages that provide abbreviated function declarations, we basically have a section that introduces names followed by a body that uses those names:

| Expression                                         | Language                |
|----------------------------------------------------|-------------------------|
| <code> e  e &lt; 0</code>                          | Rust                    |
| <code>e -&gt; e &lt; 0</code>                      | Java                    |
| <code>e =&gt; e &lt; 0</code>                      | C# 3, JavaScript, Scala |
| <code>\e -&gt; e &lt; 0</code>                     | Haskell                 |
| <code>{  e  e &lt; 0 }</code>                      | Ruby                    |
| <code>{ e in e &lt; 0 }</code>                     | Swift                   |
| <code>{ e -&gt; e &lt; 0 }</code>                  | Kotlin                  |
| <code>fun e -&gt; e &lt; 0</code>                  | F#, OCaml               |
| <code>lambda e: e &lt; 0</code>                    | Python                  |
| <code>fn e -&gt; e &lt; 0 end</code>               | Elixir                  |
| <code>[](int e){ return e &lt; 0; }</code>         | C++                     |
| <code>func(e int) bool { return e &lt; 0 }</code>  | Go                      |
| <code>delegate (int e) { return e &lt; 0; }</code> | C# 2                    |

On the plus side, C++ is not the longest (although note that C# 2's anonymous methods were very long and then they introduced the much terser lambdas in C# 3).

But the interesting case I wanted to discuss here is those languages that support placeholder expressions:

| Expression                        | Language                                 |
|-----------------------------------|------------------------------------------|
| <code>_ &lt; 0</code>             | Scala (and Boost.HOF)                    |
| <code>_1 &lt; 0</code>            | Boost.Lambda (and other Boost libraries) |
| <code>#(&lt; % 0)</code>          | Clojure                                  |
| <code>&amp;(&amp;1 &lt; 0)</code> | Elixir                                   |
| <code>{ \$0 &lt; 0 }</code>       | Swift                                    |
| <code>{ it &lt; 0 }</code>        | Kotlin                                   |

There's a lot of variety in these placeholder lambdas. Some languages number their parameters and let you write whatever you want (Swift starts numbering at 0, Elixir at 1, Clojure also at 1 but also provides % as a shorthand), Kotlin only provides the special variable it to be the first parameter and has no support if you want others.

Scala is unique in that it only provides \_, but that placeholder refers to a different parameter on each use. So `_ > _` is a binary predicate that checks if the first parameter is greater than the second. Boost.HOF does the same with its unnamed placeholders [[boost.hof.unnamed](#)].



Now, for this particular example, we also have library solutions available to us, and have for quite some time. There are several libraries *just in Boost* that allow for either `_1 < 0` or `_ < 0` to mean the same thing as illustrated above. Having placeholder lambdas is quite popular precisely because there's no noise; when writing a simple expression, having to deal with the ceremony of introducing names and dealing with the return type is excessive. For instance, to implement [P2321R1]'s `zip`, you need to dereference all the iterators in a tuple (this uses Boost.Lambda2 [boost.lambda2]):

| Regular Lambda                                                                                  | Placeholder Lambda                       |
|-------------------------------------------------------------------------------------------------|------------------------------------------|
| <pre>tuple_transform([](auto&amp; it) -&gt; decltype(auto) {     return *it; }, current_)</pre> | <pre>tuple_transform(*_, current_)</pre> |

The C++ library solutions are fundamentally limited to operators though. You can make `_1 == 0` work, but you can't really make `_1.id() == 0` or `f(_1)` work. As a language feature, having member functions work is trivial. But having non-member functions work is... not.

In the table of languages that support placeholder lambdas, four of them have *bounded* expressions: there is punctuation to mark where the lambda expression begins and ends. This is due to a fundamental ambiguity: what does `f(%)` mean? It could either mean invoking `f` with a unary lambda that returns its argument, i.e. `f(e => e)`, or it could mean a unary lambda that invokes `f` on its argument, i.e. `e => f(e)`. How do you know which one is which?

Scala, which does not have bounded placeholder expressions, takes an interesting direction here:

| Expression                 | Meaning                            |
|----------------------------|------------------------------------|
| <code>f(_)</code>          | <code>x =&gt; f(x)</code>          |
| <code>f(_, 1)</code>       | <code>x =&gt; f(x, 1)</code>       |
| <code>f(_ + 2)</code>      | <code>x =&gt; f(x + 2)</code>      |
| <code>f(1, 2, g(_))</code> | <code>f(1, 2, x =&gt; g(x))</code> |
| <code>f(1, _)</code>       | <code>x =&gt; f(1, x)</code>       |
| <code>f(1, _ + 1)</code>   | <code>f(1, x =&gt; x + 1)</code>   |
| <code>f(g(_))</code>       | <code>f(x =&gt; g(x))</code>       |
| <code>1 + f(_)</code>      | <code>x =&gt; 1 + f(x)</code>      |

I'm pretty sure we wouldn't want to go that route in C++, where we come up with some rule for what constitutes the bounded expression around the placeholder. Plus there are limitations here in the kind of expressions that you can represent, which seems like people would run into fairly quickly.

But also more to the point, when the placeholder expression refers to (odr-uses) a variable, we need to capture it, and we need to know *how* to capture it.

So a C++ approach to placeholder lambdas might be: a *lambda-introducer*, followed by some token to distinguish it from a regular lambda (e.g. `:` or `=>`), followed by a placeholder expression. That is, the placeholder lambda for the negative example might be `[]: %1 < 0` or just `[]: % < 0`. At 9 characters,

this is substantially shorter than the C++ lambda we have to write today (26 characters), and this is about as short as a C++ lambda gets (the dereference example would be 7 characters as compared to 46). And while it's longer than the various library approaches (Boost.HOF's `_ < 0` is just 5), it would have the flexibility to do anything. Probably good enough.

Alternatively could do something closer to Elixir and wrap the placeholder lambda expression, so something like: `[] %(%1 < 0)`.

For more on the topic of placeholder lambdas in C++, consider [vector<bool>](#)'s blog posts "Now I Am Become Perl" [[vector-bool](#)] and his macro-based solution to the problem [[vector-bool-macro](#)].

Getting back to the topic, when I talked originally about:

```
x |> f(y, %) // means f(y, x)
```

I noted that `f(y, %)` kind of serves as language bind expression, which itself justifies having the model support `x |> f`. But it may also justify splitting the above expression in two:

```
// some version of lambda placeholder syntax
auto fy = [=] %(f(y, %1));
auto fy = [=] %(f(y, %));
auto fy = [=]: f(y, %1);
auto fy = [=]: f(y, %);

// at which point it's just a lambda
fy(x) // evaluates to f(y, x)
```

Although note that with the lambda, you have to capture `y`, whereas with the pipeline expression, there was no intermediate step.

To return to the table I showed earlier, a placeholder-lambda that checks if its parameter is negative would range from the shorter option of `[]: % < 0` (9 characters) to `[] %(% < 0)` (11 characters). This would put us right in line with Rust (9), Java, JavaScript, C#, Scala (all 10), and Haskell (11). Although admittedly being able to name the parameters would be ideal.

## § 4.1 Partial Application

The placeholder model allows is to replace this ranges code (that requires non-trivial library machinery to work, and tends to give fairly poor diagnostics if some calls don't work out):

```
auto v = r | views::transform(f) | views::filter(g);
```

with this pipeline code (that requires no library machinery at all, and thus provide much better diagnostics, at the cost of a few extra characters):

```
auto v = r |> views::transform(%, f) |> views::filter(%, g);
```

But ranges has one more trick up its sleeve. You can do this (also with library machinery):

```
auto fg = views::transform(f) | views::filter(g);
auto v = r | fg;
```

If we write `fg` as a normal lambda using placeholders, it would look like this:

```
auto fg = [](auto&& r) { return FWD(r) |> views::transform(%, f) |>
    views::filter(%, g); };
auto v = r |> fg(%);
```

But that's not... *really* equivalent, since we don't have the constraints on the lambda right. This probably doesn't end up mattering in practice, but it'd still be nice to actually be equivalent.

With placeholder lambdas, that could be:

```
auto fg = [=] %(%1 |> views::transform(%, f) |> views::filter(%, g));
auto v = r |> fg(%);
```

Or... could it? This is actually kind of problematic, since we have two different kinds of placeholders going on here: the one to mark the lambda parameter (`%1`) and the one to mark the pipeline placeholder (`%`). This approach seems unlikely to be viable. Even if the compiler could parse it properly, could a human?

This is the sort of example that makes me hesitant about placeholder lambdas. Alternatively, the goal of a placeholder lambda was never to subsume all of the functionality of real lambdas. Just be better for simple cases. Like being able to replace:

```
employees |> views::filter(%, [](auto&& e){ return e.name() == "Tim"; });
```

with:

```
employees |> views::filter(%, [] %(%.name() == "Tim"));
```

## § 5 Choice of Placeholder

One of the important, and most readily visible, decisions in a placeholder-based design is choosing what token to use as the placeholder.

There are a few important criteria for a placeholder:

- it needs to be sufficiently terse. While `__PLACEHOLDER__`, as a reserved identifier, is something that we could use without breaking any code (or, rather, without caring whether we break any code), it would be a terrible choice of placeholder. I would be very disappointed if I ever have to type that twice. Not twice per expression, twice... ever.
- it needs to stand out as being clearly a placeholder, as the rules around placeholders are quite distinct from the rules around other language facilities. Readers need to recognize this distinction. Taking tokens that are already commonly used for multiple different operations (for instance, `*`, `<`, `>`, or `static`) would inhibit the ability to readily recognize placeholders.

This paper uses `%` as a placeholder choice of placeholder. This has prior art in Clojure, and you could also think of it as the placeholder syntax from `printf`. It's a fine choice of placeholder.

But it runs into the problem that `%` is a valid binary operator. This means that you might have code like:

```
// 10 % (x % 2)
auto y = x |> % % 2 |> 10 % %;
```

While % is not a super common binary operator, this is still... not great. Using %% as a placeholder might avoid this issue a bit, since at least the uses of the placeholder (%%) and the operator (%) are visually distinct, at the cost of an extra character for all uses.

An alternative token that is also pre-existing as a binary operator is ^ (or ^^, for similar reasoning as above), which has the added benefit of being visually appealing. If your pipeline is written vertically:

```
x |> views::transform(^, f)
  |> views::filter(^, g)
  |> views::chunk(^, 4)
```

then it looks like the placeholder is pointing to the expression that it refers to. Cute. Maybe the placeholder should be: ^\_^

Other characters that clash with binary operators are probably too commonly used as operators to even merit consideration (like > or <, which amusingly can't even be improved by duplication, since those are still operators).

One non-operator character is ?. This would only clash when piping into a conditional expression, which as described [earlier](#), is only valid as:

```
x |> ? ? y : z
```

This doesn't look great, but also is pointless to write since it doesn't buy you anything over x ? y : z. But even if this is viable, I question the aesthetics of using ? as a placeholder like this. It's just not my favorite.

This all suggests that we should try to pick a placeholder that is outside of the realm of existing C++ operators, to avoid clashes.

A potentially obvious choice is \_ (and then \_1 for a parameter for placeholder-lambdas). This is frequently used as a placeholder already, particularly \_1, so users already have a familiarity with it and a (correct) expectation of what it might mean. [\[P1110R0\]](#) also covers this thoroughly, but also points out the problem with [the \\_\\_macro in gnu gettext](#). This wasn't a deal-breaker for using \_ as a placeholder in *declarations* but does cause a problem here, since one potential place to use a placeholder would be:

```
f |> _("text")
```

which would suddenly be translated instead and become invalid, rather than invoking f("text").

The follow-up to \_ is, of course, \_\_. This is... fine. It'd be nice to have a single-character placeholder though.

# would be an interesting choice, as it is technically available as long as you don't start a line with a placeholder. This is very easy to avoid doing, since the use of |> chaining practically begs for lines to start with |>.

Another option is `$`. This character was recently added to the C basic character set for C23 [C-N2701], there is a proposal to do the same for C++ [P2558R1], with exploration of viability thereof [P2342R0]. `$` would be an excellent choice for placeholder: it has no conflicts, it clearly stands out, and is usable in lambdas as well (`$1` for the first parameter, etc.). It also has prior art in similar positions (Hack’s placeholder is `$$`, but also Bash and Awk use `$n` to refer to the *n*th parameter).

`@` is similarly a potentially-new character to use here, that is just as viable. It doesn’t clash with existing code (since such code can’t even exist yet), though aesthetically it doesn’t seem as good as `$`, particularly with the history.

A completely different direction would be to introduce a context-sensitive keyword as that placeholder. Something like Kotlin’s `it: x |> f(it, y) |> g(it, z)`. I’m not sure this is better than any of the above options, since the identifier fails to stand out (especially if we go down the [language bind](#) route, since it’s critical that it’s clear to the human whether the expression contains a placeholder or not).

To summarize, I think a potential set of options is:

| Placeholder       | Notes                                                                 |
|-------------------|-----------------------------------------------------------------------|
| <code>%</code>    | Prior art in Clojure, <code>printf</code> . Clashes with modulus      |
| <code>%%</code>   | Clashes less with modulus                                             |
| <code>^</code>    | Points up to the previous expression. Clashes with xor                |
| <code>^^</code>   | Clashes less with xor                                                 |
| <code>#</code>    | Not a C++ operator, but could have poor interaction with preprocessor |
| <code>—</code>    | Reserved token, closest thing to <code>_</code> , which isn’t viable  |
| <code>\$</code>   | Prior art in Bash/Awk/etc, brand new (potential) token, no clashing   |
| <code>\$\$</code> | Prior art in Hack, brand new (potential) token, no clashing           |
| <code>@</code>    | Brand new (potential) token, no clashing                              |
| <code>@@</code>   | Brand new (potential) token, no clashing                              |

My personal preference of these is `$`. It’s new, doesn’t clash with anything, is a single character, stands out, and could have clear meaning. And I think a single-character placeholder would be better than a two-character one.

## § 6 Disposition

My current view is that the best choice of design is:

- the [placeholder](#) model (with mandatory placeholder)
- the [precedence](#) show be very low: just above assignment.
- [multiple uses](#) of placeholder should evaluate the left-hand argument once and provide it as an lvalue to each placeholder, and [at least one use](#) has to be evaluated.
- the [choice of placeholder](#) should be `$`: it’s new, doesn’t clash with anything, is only a single character, stands out with clear meaning, and has prior art in this space.

We should explore [placeholder lambdas](#) as an option for terser lambdas. There doesn't seem to be a real direction for having terser lambdas, so this may be it.

## § 6.1 Is This Worth It?

This paper, like the `|>` paper before it, focused on Ranges for its examples. The goal was to take code like this (which today relies on involved library machinery, which has costs in compile time and poor diagnostics):

```
r | views::transform(f) | views::filter(g)
```

and replace it with a version that doesn't require any library machinery, will compile faster, and will be able to provide better diagnostics (since adapters could be written to *just* be binary functions, rather than have to deal with partial application):

```
r |> views::transform($, f) |> views::filter($, g)
```

Ranges isn't the only motivation of course, such a general expression-rewrite facility would be broadly useful.

But this isn't the only way to solve this problem. Other languages approach this differently. In Rust (or Swift), we would define a trait (or protocol) named `viewable_range`, which would have, as associated functions, all of the adapters. The benefit of doing so would once a type is a `viewable_range`, that this would work:

```
r.transform(f).filter(g)
```

At multiple points in this paper, I've commented about a few characters extra here and there between the different designs. But the above is only 24 characters, while the actual Ranges code is 42 (and would be 52 if I wrote the namespaces as `std::views::` instead). With a facility like Rust traits (or Swift protocols), getting the above to work would also take no machinery and have all of the other benefits I'm describing here.

Now, a pipeline operator with placeholders is more *flexible* - it allows all sorts of interesting possibilities, particularly when it comes to piping into somewhere other than the first parameter, or simply being used to express evaluation order in a way that is clearer than uses parentheses. But the question is: if we had traits, would I have even written this paper? After all, there's a difference between being annoyed at the entirety of Ranges and... being annoyed at, specifically, `zip_transform`.

I think that thought is a good one with which to end this paper.

## § 7 References

[boost.hof.unnamed] Paul Fultz II. 2016. Boost.HOF unnamed placeholder.

<https://www.boost.org/doc/libs/master/libs/hof/doc/html/include/boost/hof/placeholders.html#unnamed-placeholder>

- [boost.lambda2] Peter Dimov. 2020. Lambda2: A C++14 Lambda Library.  
<https://www.boost.org/doc/libs/master/libs/lambda2/doc/html/lambda2.html>
- [C-N2701] Philipp Klaus Krause. 2021. @ and \$ in source and execution character set.  
<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2701.htm>
- [javascript.pipeline] TC39. 2019. Proposals for |> operator.  
<https://github.com/tc39/proposal-pipeline-operator/>
- [P0145R3] Gabriel Dos Reis, Herb Sutter, Jonathan Caves. 2016-06-23. Refining Expression Evaluation Order for Idiomatic C++.  
<https://wg21.link/p0145r3>
- [P1110R0] Jeffrey Yasskin, JF Bastien. 2018-06-07. A placeholder with no name.  
<https://wg21.link/p1110r0>
- [P1371R3] Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. 2020-09-15. Pattern Matching.  
<https://wg21.link/p1371r3>
- [P2011R0] Barry Revzin, Colby Pike. 2020-01-07. A pipeline-rewrite operator.  
<https://wg21.link/p2011r0>
- [P2011R1] Barry Revzin, Colby Pike. 2020-04-16. A pipeline-rewrite operator.  
<https://wg21.link/p2011r1>
- [P2321R1] Tim Song. 2021-04-11. zip.  
<https://wg21.link/p2321r1>
- [P2342R0] Corentin Jabot. 2021-03-25. For a Few Punctuators More.  
<https://wg21.link/p2342r0>
- [P2558R1] Steve Downey. 2022-05-11. Add @, \$, and ` to the basic character set.  
<https://wg21.link/p2558r1>
- [vector-bool] Colby Pike. 2018. Now I Am Become Perl.  
<https://vector-of-bool.github.io/2018/10/31/become-perl.html>
- [vector-bool-macro] Colby Pike. 2021. A Macro-Based Terse Lambda Expression.  
<https://vector-of-bool.github.io/2021/04/20/terse-lambda-macro.html>